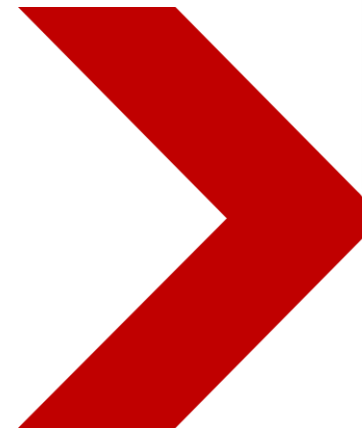


Java程序设计基础

Java Programming Fundamentals

Polymorphism



多态的概念

- 多态 (polymorphism) 是编程语言核心概念
- 为不同数据类型的实体提供统一接口，使相同操作在不同对象上引发差异化行为。
- Java中的多态
 - 方法覆写/重写 (overriding) 产生了父子同名的方法
 - 当父类引用指向子类对象的时候产生多态

多态示例

• **Father f = new Son();**

```
class Father{
    public void show() {System.out.println("show father");}
}
class Son extends Father{                //1. 继承关系
    public void show() {                  //2. 方法重写
        System.out.println("show son");
    }
}
public class PolymorphicDemo01 {
    public static void main(String[] args) {
        Father f1 = new Son();            // 3. 父类引用指向子类对象
    }
}
```

> Cat & Animal

- Cat继承自Animal, 出现Animal a=new Cat();
- 调用父类对象中的方法和变量时, 到底是使用父类的呢还是子类的呢?

```
class Animal{
    static String name = "animal";
    int num = 1;
    public static void sleep() {
        System.out.println("animal sleep");
    }
    public void run() {
        System.out.println("animal run");
    }
}
Animal a=new Cat();
a.name a.num a.sleep() a.run() a.highJump()
```

```
class Cat extends Animal{
    static String name = "cat";
    int num = 2;
    public static void sleep() {
        System.out.println("cat sleep");
    }
    public void run() {
        System.out.println("cat run");
    }
    public void highJump() {
        System.out.println("high jump");
    }
}
```

向上转型和向下转型

- **向上转型：**子类对象 —> 父类对象
 - 对于向上转型，程序会自动完成，格式：
 - 对象向上转型：父类 父类对象 = 子类实例；
 - `Father f=new Son();`
- **向下转型：**父类对象 —> 子类对象
 - 对于向下转型时，必须明确的指明要转型的子类类型，格式：
 - 对象向下转型：子类 子类对象 = (子类)父类实例；
 - `Son s=(Son)f; //要求f必须是能够转换为Son`



到底调用的父类还是子类?

```
class Animal{
    public void eat() {}
}
class Dog extends Animal{
    public void eat() {}
    public void lookDoor() {}
}
class Cat extends Animal{
    public void eat() {}
    public void catchMouse() {}
}
```

```
public class PolymorphicDemo01 {
    public static void main(String[] args) {
        Animal a=new Dog();
        a.eat();
        a.lookDoor();

        Dog d=(Dog)a;
        d.eat();
        d.lookdoor();

        a=new Cat();
        a.eat();
        a. catchMouse();

        Dog dd= (Dog) a;
        dd.eat();
        dd.lookDoor();

    }
}
```

设计一个方法，要求此方法可以接收A类的任意子类对象，并调用方法。

```
class A{// 定义父类A
    public void fun1() { // 定义fun1()方法
        System.out.println("A") ;
    }
};
class B extends A{
    public void fun1() { // 此方法被子类覆写了
        System.out.println("B") ;
    }
    public void fun2() {
        System.out.println("B --> EX") ;
    }
};
class C extends A{
    public void fun1() { // 此方法被子类覆写了
        System.out.println("C") ;
    }
    public void fun3() {
        System.out.println("C --> EX") ;
    }
};
```

```
public class PolDemo04{
    public static void main(String
    asrgs[]){
        fun(new B()) ; // 传递B的实例
        fun(new C()) ; // 传递B的实例
    }
    public static void fun(B b){
        b.fun1() ;
    }
    public static void fun(C c){
        c.fun1() ;
    }
};
```

```
public static void fun(A a){
    a.fun1() ; //使用多态，简化代码
}
```

抽象类的概念

- 抽象类用来描述一种类型应该具备的基本特征与功能。
 - 抽象类不实现具体功能。
 - 具体实现由子类通过方法重写来完成。
- 例如：犬科类动物均会吼叫
 - 狼和狗都属于犬科，但是吼叫的细节不同。
 - 犬科类规定吼叫功能，但不实现。
 - 狼和狗类通过重写实现吼叫的具体细节。

抽象类的特点

- 抽象类和抽象方法必须用关键字**abstract**修饰
- 抽象类中不一定有抽象方法,但是有抽象方法的类一定是抽象类
- **抽象类不能实例化**
- 抽象类的子类
 - 可以是一个抽象类。
 - 也可以是一个具体类。这个类必须**重写抽象类中的所有抽象方法**。

抽象类示例

```
abstract class Animal { //动物类
    public abstract void eat(); //定义一个抽象方法
}
class Dog extends Animal { //Dog类
    @Override
    public void eat() {System.out.println("我实现了父类方法, 狗吃肉");}
}
class Cat extends Animal { //Cat类
    @Override
    public void eat() {System.out.println("我实现了父类方法, 猫吃鱼");}
}
public class AnimalTest { //测试类
    public static void main(String[] args) {
        Animal a1 = new Dog();  a1.eat();
        System.out.println("-----");
        Animal a2 = new Cat();  a2.eat();
    }
}
```

抽象类的特点

- 抽象类的成员特点
 - 成员变量：既有变量，也有常量
 - 构造方法：有构造方法
 - 成员方法：既有抽象，也有非抽象
- 抽象类的问题
 - 抽象类有构造方法，不能实例化，那么构造方法有什么用？
 - 用于子类访问父类数据的初始化
 - 一个类如果没有抽象方法,却定义为了抽象类，有什么用？
 - 为了不让创建对象

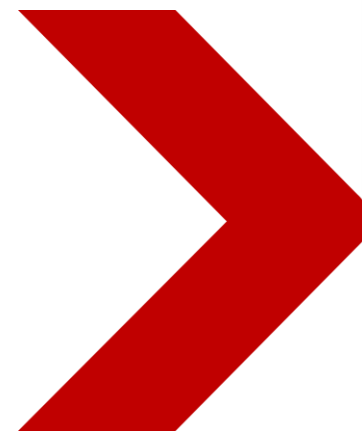
注意

- **abstract 一定不能和 private, static, final 同时使用。**
 - 抽象方法只有方法头，没有方法体，故无需实现。
 - 抽象方法的意义在于需要子类实现。private修饰后为私有，子类不能访问，故不能同时使用。
 - final用于类名前，表示类不可被继承；final用于变量前，表示它是只能一次赋值的变量，如果初始化了那就是常量，也不可被改变。和abstract搭配无意义(final不能被重写，而abstract必须重写，不能同时使用)
 - static修饰的是静态方法，可以直接被类调用；而abstract修饰的类中只有方法名，无方法体，不能被直接调用，故不能同时修饰一个类或方法。

Java程序设计基础

Java Programming Fundamentals

Interface



场景引入

- 你是杂技团驯兽师

- 教猫猫钻火圈
- 教小狗做算术

- 特点

- 这些技能不是猫狗与生俱来的，而是后天训练的。
- 可能还会教其他小动物也学会这项技能

- 如何实现技能复用

- 引入接口
- 对学会这项技能的动物而言只需要实现这个接口就可以。

- 将Interface抽象出来

Interface定义

- 接口可以理解为一种特殊的类，里面全部是由全局常量和公共的抽象方法所组成。接口的定义格式如下：

```
interface 接口名称{  
    全局常量;  
    抽象方法;  
}
```

- 成员变量：只能是常量，默认修饰符 `public static final`
- 构造方法：没有，因为接口主要是扩展功能的，而没有具体存在
- 成员方法：只能是抽象方法，默认修饰符 `public abstract`

代码示例

完整格式

```
interface IA{  
    public static final String COLLEGE = "悉尼学院" ;// 定义全局常量  
    public abstract void print() ;                // 定义抽象方法  
    public abstract String getInfo() ;              // 定义抽象方法  
}
```

简化格式

```
interface IA{  
    String COLLEGE = "悉尼学院" ;  
    void print() ;  
    String getInfo() ;  
}
```

JDK1.8以后在接口中可以定义default和static方法

接口的实现--- implements

- 一个子类可以实现多个接口

```
class 子类 implements 接口A,接口B,...{  
    }  
}
```

- 本质还是继承（多重继承）

接口实现代码示例

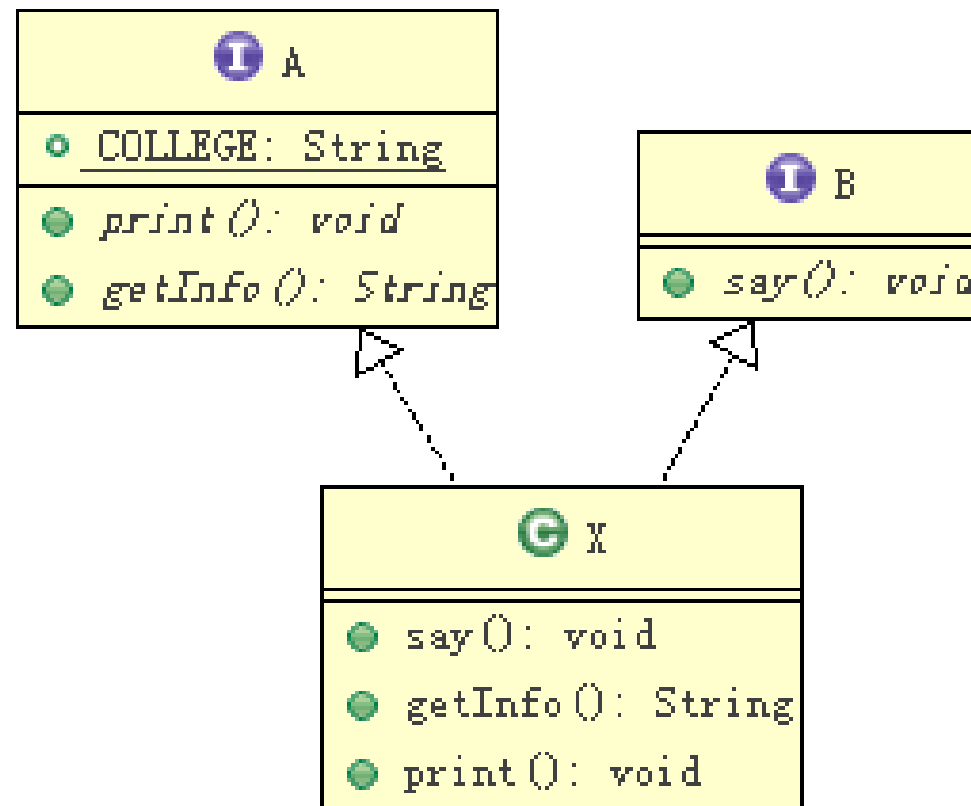
```
interface A{
    public static final String COLLEGE = "软件学院" ;// 定义全局常量
    public abstract void print() ;                // 定义抽象方法
    public abstract String getInfo() ;            // 定义抽象方法
}

interface B{                                     // 定义接口B
    public abstract void say() ;                 // 定义抽象方法
}

class X implements A,B{                         // 子类同时实现两个接口
    public void say() {                          // 覆写B接口中的抽象方法
        System.out.println("Hello World!!!");
    }
    public String getInfo() {                    // 覆写A接口中的抽象方法
        return "HELLO";
    }
    public void print() {                        // 覆写A接口中的抽象方法
        System.out.println("学院: " + COLLEGE);
    }
}
```

> 代码解释

- X类同时实现了A、B两个接口
- X类中就必须同时覆写完两个接口中的全部抽象方法。





继承+接口: class 子类 extends 抽象类 implements 接口A,接口B,...{}



```
interface A{
    public static final String COLLEGE = "软件学院" ;// 定义全局常量
    public abstract void print() ;                // 定义抽象方法
    public abstract String getInfo() ;            // 定义抽象方法
}

abstract class B{                                // 定义抽象类 B
    public abstract void say() ;
}

class X extends B implements A{                  // 子类同时实现接口
    public void say() {                          // 覆写抽象类B中的抽象方法
        System.out.println("Hello World!!!");
    }
    public String getInfo() {                    // 覆写接口A中的抽象方法
        return "HELLO";
    }
    public void print() {                        // 覆写接口A中的抽象方法
        System.out.println("学院: " + COLLEGE);
    }
}
```

java中允许一个抽象类实现多个接口

```
interface A{  
    public static final String COLLEGE = "软件学院" ;// 定义全局常量  
    public abstract void print() ;                // 定义抽象方法  
    public abstract String getInfo() ;            // 定义抽象方法  
}  
  
abstract class B implements A{                    // 定义抽象类，实现接口  
    public abstract void say() ;                  // 此时抽象类中存在三个抽象方法  
}  
  
class X extends B{                                // 子类继承抽象类  
    public void say() {                          // 覆写抽象类B中的抽象方法  
        System.out.println("Hello World!!!");  
    }  
    public String getInfo() {                    // 覆写抽象类B中的抽象方法  
        return "HELLO";  
    }  
    public void print() {                        // 覆写抽象类B中的抽象方法  
        System.out.println("学院: " + COLLEGE);  
    }  
}
```



接口的继承

- 一个接口不能继承一个抽象类，但是却可以通过extends关键字同时继承多个接口，实现接口的多继承。
- interface 子接口 extends 父接口A,父接口B,...{ }

示例

```
interface A{                                     // 定义接口A
    public static final String COLLEGE = "悉尼学院" ;// 定义全局常量
    public abstract void printA() ;              // 定义抽象方法
}

interface B{                                     // 定义接口B
    public void printB() ;                       // 定义抽象方法
}

interface C extends A,B{                       // 定义接口C, 同时继承接口A、B
    public void printC() ;                       // 定义抽象方法
}

class X implements C{                           // 子类实现接口C
    public void printA() {                       // 覆写接口A中的printA()方法
        System.out.println("A、Hello World") ;
    }
    public void printB() {                       // 覆写接口B中的printB()方法
        System.out.println("B、Hello China") ;
    }
    public void printC() {                       // 覆写接口C中的printC()方法
        System.out.println("C、Hello sstc") ;
    }
}
```

Abstract class vs. Interface

No.	区别点	抽象类	接口
1	定义	abstract定义的类	interface定义
2	组成	构造方法、抽象方法、普通方法、常量、变量	常量、抽象方法 (1.8以后可用default或static方法)
3	使用	子类继承抽象类 (extends)	子类实现接口 (implements)
4	关系	抽象类可以实现多个接口	接口不能继承抽象类, 但允许继承多个接口
5	常见设计模式	模板设计	工厂设计、代理设计
6	对象	都通过对象的多态性产生实例化对象	
7	局限	抽象类有单继承的局限	接口没有此局限
8	实际	作为一个模板	是作为一个标准或是表示一种能力
9	选择	如果抽象类和接口都可以使用的话, 优先使用接口, 因为避免单继承的局限	

instanceof关键字

- 使用instanceof关键字判断一个对象到底是那个类的实例。
- 对象 instanceof 类 //返回boolean类型
- 在进行对象向下转型之前验证，避免类型转换异常的出现。

instanceof 示例

```
class A {  
    public void fun1() {                // 定义fun1()方法  
        System.out.println("A --> public void fun1(){}");  
    }  
};  
class B extends A {                    // 子类通过extends继承父类  
    public void fun1() {                // 覆写父类中的fun1()方法  
        System.out.println("B --> public void fun1(){}");  
    }  
    public void fun2() {                // 子类自己定义的方法  
        System.out.println("B --> public void fun3(){}");  
    }  
};  
public class InstanceofDemo01 {  
    public static void main(String[] args) {  
        A a1 = new B();                // 通过向上转型实例化A类对象  
        System.out.println("A a1 = new B(): " + (a1 instanceof A));  
        System.out.println("A a1 = new B(): " + (a1 instanceof B));  
        A a2 = new A();                // 通过A类的构造实例化本类对象  
        System.out.println("A a2 = new A(): " + (a2 instanceof A));  
        System.out.println("A a2 = new A(): " + (a2 instanceof B));  
    }  
}
```

instanceof 可在向下转型前进行验证

```
public class InstanceofDemo02 {  
    public static void main(String[] args) {  
        fun(new B()) ;           // 传递B类实例，产生向上转型  
        fun(new C()) ;           // 传递C类实例，产生向上转型  
    }  
    public static void fun(A a) { // 此方法可以分别调用各自子类单独定义的方法  
        a.fun1() ;  
        if(a instanceof B) {           // 判断是否是B类实例  
            B b = (B) a ;               // 进行向下转型  
            b.fun2() ;                 // 调用子类自己定义的方法  
        }  
        if(a instanceof C) {           // 判断是否是C类实例  
            C c = (C) a ;               // 进行向下转型  
            c.fun3() ;                 // 调用子类自己定义的方法  
        }  
    }  
}
```